

Scaled Dot-Product Attention, Worked Out by Hand

THREE STEPS, ONE QUADRATIC TENSOR, SIX ROUND TRIPS — A COMPLETE NUMERICAL WALK-THROUGH

What this document does. It takes the textbook attention formula $O = \text{softmax}(QK^\top/\sqrt{d_k})V$ and computes it, by hand, on a deliberately tiny problem (3 tokens, $d_k = 2$) so that every number can be checked on paper with a ✓. It then *derives* why the score is divided by $\sqrt{d_k}$ — not as a convention, but from the variance of a dot product — and finally accounts for the memory this clean formula actually costs at production scale: the quadratic $N \times N$ score tensor and the *six HBM round trips* a naive implementation pays per layer. Every figure is produced by the reference script in Appendix A (`m03_t01_scaled_dot_product_attention_engine.py`).

The one idea. The attention formula is three matrix operations. It is cheap in FLOPs and ruinous in *memory*: the intermediate S and P matrices are each $N \times N$, exist only to be consumed by the next step, and at $N = 6200$ across 64 heads and 80 layers would demand hundreds of gigabytes of HBM the GPU does not have. That tension is the entire motivation for FlashAttention (Topic 2).

CONCEPTS COVERED, IN ORDER

the three steps $S \rightarrow P \rightarrow O$; row-wise softmax with the max-subtraction trick; a fully worked 3-token example with a hand-check; the $\sqrt{d_k}$ derivation from dot-product variance; why an unscaled softmax saturates; the $O(N^2)$ score-memory blocker; and the six HBM round trips of naive attention.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The formula under study	2
2	Step 1 — A tiny problem we can fully check	2
3	Step 2 — Row-wise softmax	3
4	Step 3 — The output $O = PV$	3
5	Step 4 — <i>Why</i> divide by $\sqrt{d_k}$?	3
6	Step 5 — The quadratic-memory blocker	4
7	Step 6 — The six HBM round trips	5
8	The argument, at a glance	5
8.1	Equation cheat-sheet	6

Appendix A — Reference implementation

6

1 The formula under study

Scaled dot-product attention (Vaswani et al., 2017) maps three matrices $Q, K, V \in \mathbb{R}^{N \times d_k}$ — queries, keys, values for N tokens of head dimension d_k — to an output $O \in \mathbb{R}^{N \times d_k}$ in three steps.

Scaled dot-product attention

$$\underbrace{S = \frac{QK^\top}{\sqrt{d_k}}}_{[N \times N] \text{ scores}} \quad \underbrace{P = \text{softmax}(S)}_{[N \times N] \text{ probabilities, row-wise}} \quad \underbrace{O = PV}_{[N \times d_k] \text{ output}}$$

Intuition

Each query row asks: “of all the tokens I can see, how much should I listen to each one?” Step 1 scores every query against every key by similarity (a dot product). Step 2 turns each row of scores into a probability distribution over the keys. Step 3 forms each output as that probability-weighted average of the value vectors. The whole operation is “soft lookup”: a differentiable, weighted dictionary read.

The shapes are the whole story. Q, K, V are each $[N \times d_k]$, but the *intermediate* S and P are each $[N \times N]$ — quadratic in sequence length. Steps 5–6 show why that single fact dominates the memory cost.

2 Step 1 — A tiny problem we can fully check

To make every number verifiable, fix $N = 3$ tokens and $d_k = 2$:

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 10 & 0 \\ 0 & 10 \\ 5 & 5 \end{bmatrix}.$$

Here $Q = K$ (token 1 “points” along axis 1, token 2 along axis 2, token 3 diagonally), and the values are chosen so the answer is easy to sanity-check.

Step 1: raw scores, then the scale

The raw score matrix is every query · every key:

$$S_{\text{raw}} = QK^\top = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

For example $S_{\text{raw}}[3, 3] = (1, 1) \cdot (1, 1) = 2$. Now scale by $1/\sqrt{d_k} = 1/\sqrt{2} = 0.7071$:

$$S = \frac{S_{\text{raw}}}{\sqrt{2}} = \begin{bmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 0.7071 & 0.7071 \\ 0.7071 & 0.7071 & 1.4142 \end{bmatrix}.$$

3 Step 2 — Row-wise softmax

Softmax is applied independently to each *row* of S (each query's distribution over keys). For numerical safety we subtract the row maximum before exponentiating — this changes nothing mathematically (it cancels in the ratio) but keeps every exponent ≤ 0 :

$$P_{ij} = \frac{\exp(S_{ij} - m_i)}{\sum_k \exp(S_{ik} - m_i)}, \quad m_i = \max_k S_{ik}.$$

Step 2: softmax of row 1, by hand

Row 1 is $S[1] = (0.7071, 0, 0.7071)$ with $m_1 = 0.7071$. Subtract and exponentiate:

$$\exp(S[1] - m_1) = (e^0, e^{-0.7071}, e^0) = (1, 0.4931, 1), \quad \text{sum} = 2.4931.$$

$$P[1] = \frac{(1, 0.4931, 1)}{2.4931} = (0.4011, 0.1978, 0.4011).$$

Doing all three rows:

$$P = \begin{bmatrix} 0.4011 & 0.1978 & 0.4011 \\ 0.1978 & 0.4011 & 0.4011 \\ 0.2483 & 0.2483 & 0.5035 \end{bmatrix}, \quad \text{each row sums to 1.0. } \checkmark$$

Mini-example — the max-subtraction is free

Subtracting m_i multiplies numerator and denominator by the same e^{-m_i} , so the ratio is unchanged: softmax is shift-invariant. We do it only so that $\exp(\cdot)$ never overflows — $\exp(0) = 1$ is the largest term we ever form. This exact trick is what makes the *online* softmax of Topic 2 possible.

4 Step 3 — The output $O = PV$

Each output row is the probability-weighted average of the value rows.

Step 3: output, with a hand-check of row 1

$$O = PV = \begin{bmatrix} 6.0167 & 3.9833 \\ 3.9833 & 6.0167 \\ 5.0000 & 5.0000 \end{bmatrix}.$$

Check row 1 directly:

$$\begin{aligned} O[1] &= 0.4011 (10, 0) + 0.1978 (0, 10) + 0.4011 (5, 5) \\ &= (4.011 + 0 + 2.006, 0 + 1.978 + 2.006) = (6.017, 3.983). \checkmark \end{aligned}$$

It matches the matrix product to rounding. Row 3 is exactly $(5, 5)$: token 3 sits symmetrically between tokens 1 and 2, so it averages their values evenly — a result you can see without arithmetic, and the script confirms it.

5 Step 4 — Why divide by $\sqrt{d_k}$?

The $\sqrt{d_k}$ is not cosmetic. It keeps the softmax in its responsive range as the head dimension grows.

The variance of a dot product

Take query q and key k with independent, zero-mean, unit-variance entries q_i, k_i . Their dot product is $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$. Each term $q_i k_i$ has mean $\mathbb{E}[q_i] \mathbb{E}[k_i] = 0$ and variance $\text{Var}(q_i k_i) = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = 1$. The terms are independent, so variances add:

$$\text{Var}(q \cdot k) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k, \quad \text{std}(q \cdot k) = \sqrt{d_k}.$$

So a raw score has typical magnitude $\sqrt{d_k}$. Dividing by $\sqrt{d_k}$ renormalizes the scores back to unit scale *regardless of head dimension*.

The script confirms this empirically over 200,000 random pairs at several dimensions:

d_k	$\text{Var}(q \cdot k)$ (measured)	std (measured)	$\sqrt{d_k}$
2	2.015	1.419	1.414
8	8.035	2.835	2.828
64	63.75	7.985	8.000
128	128.24	11.324	11.314

What unscaled scores do to softmax, at $d_k = 128$

With one query against four random keys at $d_k = 128$, the script draws raw logits

$$(0.94, -11.01, 8.40, -0.98) \xrightarrow{\text{softmax}} (0.0006, 0.0000, 0.9993, 0.0001),$$

a near one-hot, *saturated* distribution: one key wins everything, the gradient to the others is essentially zero. Scaling by $1/\sqrt{128}$ first gives

$$(0.083, -0.973, 0.743, -0.086) \xrightarrow{\text{softmax}} (0.242, 0.084, 0.469, 0.205),$$

a responsive distribution that still prefers the best key but keeps usable gradient on all of them. That is exactly what $\sqrt{d_k}$ buys.

Why the scale matters

Without $\sqrt{d_k}$, scores grow like $\sqrt{d_k}$, the softmax saturates toward a hard arg max, and gradients vanish — training stalls and the model cannot learn to attend softly. Dividing by $\sqrt{d_k}$ holds the logits at unit scale for any head dimension, keeping softmax in the smooth, trainable regime. It is a variance normalizer, derived in one line.

6 Step 5 — The quadratic-memory blocker

The formula is clean; its memory footprint is not. S and P are each $N \times N$ *per head*, and they exist only as scratch between steps. At Nexus scale ($N = 6200$, $H = 64$ heads, FP16, 80 layers):

$O(N^2)$ score memory at production scale

One head's S matrix:

$$N^2 \cdot 2 \text{ B} = 6200^2 \cdot 2 = 7.688 \times 10^7 \text{ B} = 76.9 \text{ MB}.$$

S and P together across all 64 heads, per layer:

$$2 \times 76.9 \text{ MB} \times 64 = 9.84 \text{ GB per layer}.$$

Across 80 layers, the throwaway attention intermediates would total

$$9.84 \text{ GB} \times 80 \approx 787 \text{ GB. } \checkmark$$

Watch out

787 GB is roughly ten H100s' worth of HBM — for *intermediates that are immediately discarded*. The memory scales as $O(N^2)$: doubling the context quadruples it. This is precisely why “long context” was impractical before 2022, and why every modern kernel (FlashAttention, xFormers, PyTorch SDPA, vLLM) refuses to materialize S and P at all.

7 Step 6 — The six HBM round trips

Even processing one head at a time (so S , P fit in HBM, not SRAM), a naive implementation moves these matrices across the slow HBM bus repeatedly. Per head, per layer:

#	HBM operation	What crosses the bus
1	read Q, K	the inputs
2	write $S = QK^\top$	$N \times N$ scores out
3	read S	$N \times N$ scores back in
4	write $P = \text{softmax}(S)$	$N \times N$ probs out
5	read P, V	$N \times N$ probs back in
6	write $O = PV$	the output

Intuition

The arithmetic of attention (the matmuls and the softmax) would fit happily in the fast on-chip SRAM. The problem is that the $N \times N$ scratch is too big for SRAM, so the kernel parks it in slow HBM and reads it back twice. Steps 2–5 are pure memory traffic on a tensor that never needed to leave the chip. FlashAttention’s whole idea (Topic 2) is to *fuse* these six trips into one — read Q, K, V once, write O once, and never write S or P at all — using the online softmax to avoid needing the full row before normalizing.

The setup for everything that follows

Attention is cheap in FLOPs and expensive in *memory movement*. The $N \times N$ score tensor is both a quadratic capacity problem (Step 5) and a repeated-traffic problem (Step 6). Topic 2 (FlashAttention) attacks the traffic; Topics 3–4 (GQA, MLA) attack the *other* attention memory hog, the KV cache, during decode.

8 The argument, at a glance

#	Step	Result
1	Tiny problem	$N = 3, d_k = 2; S_{\text{raw}} = QK^\top$
2	Softmax	$P[1] = (0.4011, 0.1978, 0.4011)$, rows sum to 1 $\checkmark$
3	Output	$O[1] = (6.017, 3.983)$, $O[3] = (5, 5)$ $\checkmark$
4	Why $\sqrt{d_k}$	$\text{std}(q \cdot k) = \sqrt{d_k}$; unscaled $\Rightarrow$ saturated softmax
5	Quadratic memory	$S, P = 9.84 \text{ GB/layer} \Rightarrow 787 \text{ GB over 80 layers}$
6	Six round trips	naive attention re-reads S, P from HBM

8.1 Equation cheat-sheet

Scores	$S = QK^\top / \sqrt{d_k}$
Softmax (row i)	$P_{ij} = \exp(S_{ij} - m_i) / \sum_k \exp(S_{ik} - m_i)$
Output	$O = PV$
Scale rationale	$\text{std}(q \cdot k) = \sqrt{d_k} \Rightarrow \text{divide by } \sqrt{d_k}$
Score memory	$2N^2H$ (dtype) per layer $[O(N^2)]$

Appendix A — Reference implementation

Every number above is produced by `m03_t01_scaled_dot_product_attention_engine.py`. Run it to reproduce the worked example, the variance demonstration, and the memory accounting; change Q, K, V or the production constants to explore other cases.

```

1  """
2  Reference implementation for: Module 03 - Topic 1
3  "Scaled Dot-Product Attention, Worked Out by Hand."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  Convention: 1 GB = 1e9 bytes, 1 MB = 1e6 bytes (decimal, the convention vendors
9  use for HBM bandwidth and model "size in GB").
10 """
11
12 import numpy as np
13
14 np.set_printoptions(precision=4, suppress=True)
15
16 # -----
17 # 0. A tiny, fully checkable attention problem: 3 tokens, d_k = 2.
18 #   These small integers are chosen so every step is verifiable by hand.
19 # -----
20 d_k = 2
21 Q = np.array([[1.0, 0.0],
22               [0.0, 1.0],
23               [1.0, 1.0]])          # [N x d_k], N = 3
24 K = np.array([[1.0, 0.0],
25               [0.0, 1.0],
26               [1.0, 1.0]])
27 V = np.array([[10.0, 0.0],
28               [0.0, 10.0],
29               [5.0, 5.0]])
30
31 N = Q.shape[0]
32 print("== 0. Inputs (N=3 tokens, d_k=2) ==")
33 print("Q =\n", Q, "\nK =\n", K, "\nV =\n", V)
34
35 # -----
36 # 1. Raw scores S = Q K^T, then scaled by 1/sqrt(d_k).
37 # -----
38 S_raw = Q @ K.T
39 scale = 1.0 / np.sqrt(d_k)
40 S = S_raw * scale
41 print("\n== 1. Scores ==")
42 print("S_raw = Q K^T =\n", S_raw)
43 print(f"scale = 1/sqrt(d_k) = 1/sqrt({d_k}) = {scale:.6f}")
44 print("S = S_raw / sqrt(d_k) =\n", S)
45

```

```

46 # -----
47 # 2. Row-wise softmax: P = softmax(S). Subtract row max for stability.
48 # -----
49 def softmax_rows(M):
50     m = M.max(axis=1, keepdims=True)
51     e = np.exp(M - m)
52     return e / e.sum(axis=1, keepdims=True)
53
54 P = softmax_rows(S)
55 print("\n== 2. Row-wise softmax P = softmax(S) ==")
56 print("P =\n", P)
57 print("row sums =", P.sum(axis=1), "(each must be 1.0)")
58
59 # -----
60 # 3. Output O = P V.
61 # -----
62 O = P @ V
63 print("\n== 3. Output O = P V ==")
64 print("O =\n", O)
65
66 # -----
67 # 4. Detailed hand-check of row 0 (query token 1).
68 # -----
69 print("\n== 4. Hand-check, query row 0 ==")
70 s0 = S[0]
71 print(f"S[0]          = {s0}")
72 m0 = s0.max()
73 e0 = np.exp(s0 - m0)
74 p0 = e0 / e0.sum()
75 print(f"max(S[0])      = {m0:.6f}")
76 print(f"exp(S[0]-max) = {e0}")
77 print(f"sum exps       = {e0.sum():.6f}")
78 print(f"P[0]           = {p0}")
79 o0 = p0 @ V
80 print(f"O[0] = P[0] V = {o0}")
81 print(f"matches O[0]? {np.allclose(o0, O[0])}")
82
83 # -----
84 # 5. WHY divide by sqrt(d_k): the dot product of two d_k-dim vectors with
85 #   iid unit-variance entries has variance d_k, i.e. std ~ sqrt(d_k).
86 #   Monte-Carlo demonstration over many random pairs at several d_k.
87 # -----
88 print("\n== 5. Why 1/sqrt(d_k): variance of q.k grows like d_k ==")
89 rng = np.random.default_rng(0)
90 for dk in [2, 8, 64, 128]:
91     q = rng.standard_normal((200000, dk))
92     k = rng.standard_normal((200000, dk))
93     dots = np.sum(q * k, axis=1)
94     print(f"  d_k={dk:4d}: var(q.k)={dots.var():8.3f}  std={dots.std():7.3f}")
95     print(f"    sqrt(d_k)={np.sqrt(dk):7.3f}")
96
97 # Effect on softmax sharpness at d_k=128, a "typical" head dim.
98 print("\n Effect on softmax (one query vs 4 keys), d_k=128:")
99 dk = 128
100 q = rng.standard_normal(dk)
101 Kk = rng.standard_normal((4, dk))
102 raw = Kk @ q
103 print(f"  unscaled logits   = {raw}")
104 print(f"  softmax(unscaled) = {softmax_rows(raw[None, :])[0]} (near one-hot, saturated)")
105 sc = raw / np.sqrt(dk)
106 print(f"  scaled logits     = {sc}")
107 print(f"  softmax(scaled)   = {softmax_rows(sc[None, :])[0]} (responsive)")
108

```

```

109 # -----
110 # 6. The quadratic memory blocker at production scale (Nexus).
111 #   S and P are each [N x N] per head.  N=6200, 64 heads, FP16, 80 layers.
112 # -----
113 print("\n== 6. Quadratic N x N score memory (Nexus scale) ==")
114 N_prod = 6200
115 H = 64
116 L = 80
117 dtype_b = 2 # FP16
118 per_head = N_prod * N_prod * dtype_b
119 print(f"S (one head) = N^2 * 2B = {N_prod}^2 * 2 = {per_head:.3e} B = {per_head/1e6:.1f} MB")
120 SP_layer = 2 * per_head * H
121 print(f"S+P, all 64 heads = 2 * {per_head/1e6:.1f}MB * {H} = {SP_layer/1e9:.2f} GB / layer")
122 print(f"across {L} layers = {SP_layer*L/1e9:.0f} GB of throwaway intermediates")
123
124 # -----
125 # 7. The "six HBM round trips" of naive attention per head per layer.
126 # -----
127 print("\n== 7. Six HBM round trips (naive attention) ==")
128 trips = ["1. read Q, K",
129          "2. write S = QK^T",
130          "3. read S",
131          "4. write P = softmax(S)",
132          "5. read P, V",
133          "6. write O = PV"]
134 for t in trips:
135     print(" ", t)
136 print(" -> S and P (the two N x N matrices) are materialized in HBM and re-read.")

```

— END OF DOCUMENT —